



SGCI

Backend — Documentação Técnica Aprofundada

API REST em Quarkus · Java 21 · Análise do código-fonte

Quarkus 3.37.1

Hibernate ORM + Panache

PostgreSQL + Flyway

SmallRye JWT

Atualizado em 12/07/2026

Sumário

1. Panorama e princípios de design
2. Configuração da aplicação (`application.properties`)
3. Inicialização e sementeira (`DataSeeder`)
4. Camada de entidades (Panache Active Record)
5. Enums de domínio
6. Camada de repositórios (DAO)
7. Camada de serviços (BO) — regras de negócio comentadas
8. Camada de controllers (JAX-RS)
9. DTOs e validação (Bean Validation)
10. Segurança — JWT em cookie, passo a passo
11. Auditoria automática por filtro de resposta
12. Tratamento de exceções
13. Migrações Flyway (V1–V10)
14. Ciclo de vida completo de uma crise (walkthrough)
15. Área do usuário (perfil COMUM): primeiro acesso, recuperação de senha e posse de crises

1. Panorama e princípios de design

O backend do SGCI é uma API REST stateless construída em **Quarkus 3.37.1 sobre Java 21**. Ele concentra toda a lógica de negócio de gestão de crises e persiste os dados em PostgreSQL. O cliente Angular apenas consome os endpoints JSON.

Arquitetura em camadas

HTTP →	Controller (JAX-RS)	valida DTO (@Valid) e autorização (@RolesAllowed)
	Service (BO)	regra de negócio, @Transactional
	Repository (DAO/Panache)	queries e acesso a dados
	Entity (Hibernate ORM)	mapeamento objeto ↔ tabela
	PostgreSQL	schema versionado por Flyway

Os comentários do próprio código nomeiam o Service como **BO (Business Object)** e o Repository como **DAO**, deixando a separação de responsabilidades explícita.

Cinco decisões de design que definem o projeto

#	Decisão	Consequência prática
1	Panache Active Record : entidades com campos <code>public</code> , sem getters/setters.	Código enxuto; acesso direto a <code>crise.status</code> .
2	Sem associações JPA : relações são colunas <code>Long</code> (ex.: <code>tipoCriseId</code>), não <code>@ManyToOne</code> .	Integridade referencial fica só no banco (FK do Flyway); nenhum cascade/lazy-loading.
3	DTOs como record : <code>*Request</code> (entrada validada) e <code>*Response</code> (saída via <code>from()</code>).	Entidades nunca vazam para a API; a senha nunca é serializada.
4	Autenticação por JWT em cookie HttpOnly (não no header Authorization).	Token inacessível ao JS → proteção contra XSS.
5	Soft-delete : DELETE marca <code>active=false</code> .	Histórico preservado; nada é apagado fisicamente.

Estrutura de pacotes

```
br.com.crisismanagement
├─ config/           DataSeeder                (bootstrap de dados)
├─ controllers/      11 resources REST
├─ dto/              records Request / Response (+ ErrorResponse padrão)
├─ entities/         10 entidades + enums/ (4)
├─ exceptions/       InvalidCredentialsException + 3 ExceptionMappers
├─ repositories/     10 DAOs Panache
├─ security/         AuthCookieFactory · AuditoriaLogFilter
└─ services/         12 BOs + PasswordGenerator (util)
```

2. Configuração da aplicação

Todo o comportamento de infraestrutura é declarado em `src/main/resources/application.properties` :

```
# App + servidor HTTP
quarkus.application.name=crisis-management-api
quarkus.http.port=8080

# Datasource PostgreSQL (porta 5433)
quarkus.datasource.db-kind=postgresql
quarkus.datasource.username=postgres
quarkus.datasource.password=postgres
quarkus.datasource.jdbc.url=jdbc:postgresql://localhost:5433/crisis_management

# Schema é do Flyway; Hibernate apenas valida (não altera tabelas)
quarkus.hibernate-orm.database.generation=validate
quarkus.flyway.migrate-at-start=true
quarkus.flyway.baseline-on-migrate=true

# JWT assinado (RS256) lido de COOKIE, não do header Authorization
mp.jwt.verify.issuer=https://crisis-management/issuer
mp.jwt.verify.publickey.location=publicKey.pem
smallrye.jwt.sign.key.location=privateKey.pem
mp.jwt.token.header=Cookie
mp.jwt.token.cookie=jwt

# CORS para o dev server Angular (credenciais on = cookie cruza origens)
quarkus.http.cors.enabled=true
quarkus.http.cors.origins=http://localhost:4200
quarkus.http.cors.methods=GET,POST,PUT,PATCH,DELETE,OPTIONS
quarkus.http.cors.headers=Content-Type,Authorization
quarkus.http.cors.access-control-allow-credentials=true

# E-mail transacional (senha temporária e link de recuperação). mock=true → só loga no console
quarkus.mailer.from=${MAIL_FROM:no-reply@sghi.local}
quarkus.mailer.host=${MAIL_HOST:smtp.gmail.com}
quarkus.mailer.port=${MAIL_PORT:587}
quarkus.mailer.mock=${MAIL MOCK:true}
app.frontend.url=${FRONTEND_URL:http://localhost:4200}
```

Combinação-chave: `database.generation=validate + migrate-at-start=true` significa que o Flyway é a única fonte de verdade do schema, e o Hibernate apenas confere, no start, que as entidades batem com as tabelas. Se divergirem, a aplicação nem sobe — um mecanismo de segurança.

3. Inicialização e sementeira — DataSeeder

Ao subir, o Quarkus dispara um `StartupEvent`. O `DataSeeder` o observa e cria, de forma **idempotente** (só se ainda não existirem), dois usuários iniciais com senha hashada em BCrypt.

```

@ApplicationScoped
public class DataSeeder {
    @Transactional
    void onStart(@Observes StartupEvent event) {
        seedAdmin();
        seedComum();
    }
    private void seedAdmin() {
        if (userRepository.findByEmail("admin@crisis.local").isPresent()) return;
        User admin = new User();
        admin.name = "Administrador";
        admin.email = "admin@crisis.local";
        admin.password = BcryptUtil.bcryptHash("Admin@123"); // nunca em texto puro
        admin.perfil = PerfilUsuario.ADMIN;
        admin.active = true;
        admin.createdAt = LocalDateTime.now();
        userRepository.persist(admin);
    }
    // seedComum(): usuario@crisis.local / Comum@123, perfil COMUM,
    // vinculado ao 1º departamento existente (ou null).
}

```

Perfil	E-mail	Senha	Departamento
ADMIN	admin@crisis.local	Admin@123	—
COMUM	usuario@crisis.local	Comum@123	1º cadastrado (CTI)

Os usuários semeados nascem com `mustChangePassword=false` (a coluna tem `DEFAULT FALSE`) — a flag de troca obrigatória vale para contas criadas depois, sem senha explícita.

4. Camada de entidades (Panache Active Record)

Todas estendem `PanacheEntityBase`, têm id `@GeneratedValue(IDENTITY)` e enums gravados como texto (`@Enumerated(EnumType.STRING)`). Repare que as FKs são `Long` puros:

```
@Entity @Table(name = "crises")
public class Crise extends PanacheEntityBase {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    public Long id;
    @Column(nullable = false, length = 200) public String titulo;
    @Column(nullable = false, length = 1000) public String descricao;
    @Enumerated(EnumType.STRING) public StatusCrise status;
    @Enumerated(EnumType.STRING) public NivelCriticidade nivel;
    public Long tipoCriseId; public Long campusId; // FK por id
    public Long cenarioId; public Long departamentoId; // (não @ManyToOne)
    public Long createdBy; // FK users: quem cadastrou (nulo pré-V10)
    public boolean active; public LocalDateTime createdAt;
}
```

Catálogo de entidades

Entidade / Tabela	Campos
User · users	id, name, email (único), password (BCrypt), perfil, departamentold?, instituicaold?, cargo?, active, mustChangePassword, resetToken?, resetTokenExpiresAt?, createdAt
Crise · crises	id, titulo, descricao, status, nivel, tipoCriseld, campusId, cenariold?, departamentold, createdBy? (quem cadastrou), active, createdAt
AcaoCrise · acoes_crise	id, descricao, tipo, criseld, usuariold, createdAt
RelatorioCrise · relatorios_crise	id, titulo, resumoOcorrido, impactosIdentificados?, acoesRealizadas?, resultadoFinal?, recomendacoes?, criseld (único), usuariold, createdAt
Instituicao · instituicoes	id, nome, sigla, active, createdAt
Campus · campi	id, nome, instituicaold, active, createdAt
Departamento · departamentos	id, nome, sigla, active, createdAt
TipoCrise · tipos_crise	id, nome, descricao?, active, createdAt
CenarioCrise · cenarios_crise	id, descricao, nivelSugerido?, tipoCriseld, departamentold, active, createdAt
AuditoriaLog · auditoria_logs	id, usuariold, usuarioNome, acao, metodoHttp, entidadeAfetada, recurso, statusResposta, dataHora

(? = coluna anulável)

5. Enums de domínio

Enum	Valores	Papel
NivelCriticidade	BAIXO · MEDIO · ALTO · CRITICO	Gravidade da crise
StatusCrise	ABERTA · EM_ANDAMENTO · RESOLVIDA · ENCERRADA	Ciclo de vida da crise
TipoAcao	CONTENCAO · COMUNICACAO · MONITORAMENTO · RESOLUCAO	Categoria de cada ação registrada
PerfilUsuario	ADMIN · COMUM	Papel de acesso — o nome vira a <i>role</i> do JWT

O comentário no código de `PerfilUsuario` é explícito: "O nome do enum é usado como 'role' (claim *groups* do JWT) para as verificações de `@RolesAllowed` ." Ou seja, adicionar um perfil novo aqui já o torna utilizável nas anotações de segurança.

6. Camada de repositórios (DAO)

Cada repositório implementa `PanacheRepository<T>`, herdando `persist`, `findByIdOptional`, `count`, `list` etc. Só os métodos específicos de consulta são escritos.

USERREPOSITORY — BUSCA POR E-MAIL E POR TOKEN DE RECUPERAÇÃO

```
public Optional<User> findByEmail(String email) {
    return find("email", email).firstResultOptional();
}

public Optional<User> findByResetToken(String resetToken) { // recuperação de senha
    return find("resetToken", resetToken).firstResultOptional();
}

public List<User> listAllOrdered() { return list("order by name"); }
```

CRISEREPOSITORY — CONTAGENS E VISÃO POR DONO

```
public List<Crise> listAllOrdered() { return list("order by createdAt desc"); }
// Crises cadastradas por um usuário específico (visão do perfil COMUM)

public List<Crise> listByCreator(Long createdBy) {
    return list("createdBy = ?1 order by createdAt desc", createdBy);
}

public long countByCreator(Long createdBy) { return count("createdBy = ?1", createdBy); }
public long countByStatus(StatusCrise status) { return count("status = ?1", status); }
public long countByNivel(NivelCriticidade nivel) { return count("nivel = ?1", nivel); }
```

AUDITORIAREPOSITORY — FILTRO DINÂMICO

O WHERE é montado conforme os parâmetros preenchidos (nulos/vazios são ignorados):

```
public List<AuditoriaLog> search(String entidade, String acao, String usuario) {
    StringBuilder query = new StringBuilder("1 = 1");
    Map<String, Object> params = new HashMap<>();
    if (entidade != null && !entidade.isBlank()) {
        query.append(" and entidadeAfetada = :entidade"); params.put("entidade", entidade);
    }
    // idem para acao (=) e usuario: lower(usuarioNome) like %...%
    query.append(" order by dataHora desc");
    return find(query.toString(), params).list();
}
```

MÉTODOS POR REPOSITÓRIO

Repository	Métodos além do CRUD Panache
UserRepository	findByEmail , findByResetToken , listAllOrdered
CriseRepository	listAllOrdered , listByCreator , countByCreator , countByStatus , countByNivel
AcaoCriseRepository	listByCrise (por crise, mais recente primeiro)
RelatorioCriseRepository	listAllOrdered , existsByCrise
AuditoriaRepository	search , distinctEntidades/Acoes/Usuarios
Departamento / TipoCrise / Instituicao / Campus	listAllOrdered (por nome)
CenarioCrise	listAllOrdered (por descrição)

7. Camada de serviços (BO) — regras de negócio

Todos `@ApplicationScoped` ; métodos de escrita são `@Transactional` . Uma técnica recorrente: como as entidades são *managed* dentro da transação, **alterar um campo já persiste** — não é preciso chamar `persist` no `update` .

7.1 CriseService — o coração do domínio

Criar: a crise sempre nasce `ABERTA` e ativa; além disso o servidor grava o **dono** (`createdBy`) a partir do usuário do JWT — é ele que habilita a "posse" das crises:

```
@Transactional
public CriseResponse create(CriseRequest request) {
    validarReferencias(request);
    User autor = usuarioAtual(); // resolvido do subject do JWT
    Crise crise = new Crise();
    crise.titulo = request.titulo(); crise.descricao = request.descricao();
    crise.nivel = request.nivel(); crise.tipoCriseId = request.tipoCriseId();
    crise.campusId = request.campusId(); crise.cenarioId = request.cenarioId();
    crise.departamentoId = request.departamentoId();
    crise.createdBy = autor.id; // dono da crise (posse)
    crise.status = StatusCrise.ABERTA; // forçado pelo servidor
    crise.active = true;
    crise.createdAt = LocalDateTime.now();
    criseRepository.persist(crise);
    return CriseResponse.from(crise, autor.name);
}
```

Listagem e detalhe são role-aware: o `ADMIN` enxerga todas as crises; o `COMUM` só as que ele mesmo cadastrou. O acesso a uma crise específica passa por um helper `getAcessivel` que lança `ForbiddenException` (403) se um `COMUM` tentar abrir a crise de outra pessoa:

```
public List<CriseResponse> listAll() {
    User atual = usuarioAtual();
    List<Crise> crises = atual.perfil == PerfilUsuario.COMUM
        ? criseRepository.listByCreator(atual.id) // COMUM: só as próprias
        : criseRepository.listAllOrdered(); // ADMIN: todas
    Map<Long,String> nomes = nomesUsuarios(crises.stream().map(c -> c.createdBy).toList());
    return crises.stream().map(c -> CriseResponse.from(c, nomes.get(c.createdBy))).toList();
}

private Crise getAcessivel(Long id) { // usado em findById / listAcoes / addAcao
    Crise crise = getOrThrow(id);
    User atual = usuarioAtual();
    if (atual.perfil == PerfilUsuario.COMUM && !atual.id.equals(crise.createdBy))
        throw new ForbiddenException("Você não tem acesso a esta crise."); // 403
    return crise;
}
```

Resolução de nomes em lote: as respostas exibem nomes legíveis — `createdByName` na crise e `usuarioNome` em cada ação — resolvidos por **uma única consulta** (`id in ?1`) sobre o conjunto de ids:

```
private Map<Long,String> nomesUsuarios(Collection<Long> ids) {
    List<Long> distintos = ids.stream().filter(Objects::nonNull).distinct().toList();
    if (distintos.isEmpty()) return new HashMap<>();
    return userRepository.list("id in ?1", distintos).stream()
        .collect(Collectors.toMap(u → u.id, u → u.name, (a, b) → a, HashMap::new));
}
```

Armadilha real (resolvida): o mapa de nomes usa `HashMap` de propósito — e **nunca** `Map.of()`. Crises anteriores à V10 têm `createdBy` nulo, e o resolvidor é consultado com `nomes.get(null)`. Um `Map.of()` (mapa imutável do JDK) lança `NullPointerException` em `get(null)`; o `HashMap` é tolerante a chave nula e apenas devolve `null`.

Registrar ação (hoje **exclusivo do ADMIN** — o endpoint `POST /{id}/acoes` passou a `@RolesAllowed("ADMIN")`): o autor sai do JWT e a crise é carregada por `getAcessivel`. A trava de departamento do `COMUM` permanece no BO como defesa em profundidade, mas na prática o `COMUM` nem alcança o método:

```
@Transactional
public AcaoCriseResponse addAcao(Long criseId, AcaoCriseRequest request) {
    Crise crise = getAcessivel(criseId);
    User autor = usuarioAtual(); // autor = subject do JWT
    // defesa em profundidade: COMUM só atuaria no próprio departamento
    if (autor.perfil == PerfilUsuario.COMUM && !crise.departamentoId.equals(autor.departamentoId))
        throw new ForbiddenException("Você só pode registrar ações em crises do seu departamento.");
    AcaoCrise acao = new AcaoCrise();
    acao.descricao = request.descricao(); acao.tipo = request.tipo();
    acao.criseId = crise.id; acao.usuarioId = autor.id;
    acao.createdAt = LocalDateTime.now();
    acaoCriseRepository.persist(acao);
    return AcaoCriseResponse.from(acao, autor.name);
}
```

Mudar status: uma crise `ENCERRADA` é terminal — não muda mais:

```
@Transactional
public CriseResponse changeStatus(Long id, StatusCrise novoStatus) {
    Crise crise = getOrThrow(id);
    if (crise.status == StatusCrise.ENCERRADA)
        throw new BadRequestException("Não é possível alterar o status de uma crise encerrada.");
    crise.status = novoStatus; // persistido pela transação
    return CriseResponse.from(crise, nomeUsuario(crise.createdBy));
}
```

7.2 RelatorioCriseService — relatório único e tardio

```
@Transactional
public RelatorioCriseResponse create(RelatorioCriseRequest request) {
    Crise crise = criseRepository.findByIdOptional(request.criseId()).orElseThrow( ... );
    if (crise.status != StatusCrise.RESOLVIDA && crise.status != StatusCrise.ENCERRADA)
        throw new BadRequestException("Só é possível gerar relatório de crises RESOLVIDAS ou ENCERRADAS.");
    if (relatorioCriseRepository.existsByCrise(crise.id))
        throw new BadRequestException("Esta crise já possui um relatório.");
    // ... preenche campos ...
    relatorio.usuarioId = Long.valueOf(jsonWebToken.getSubject());
    relatorioCriseRepository.persist(relatorio);
    return RelatorioCriseResponse.from(relatorio);
}
```

Dupla proteção contra duplicidade: checagem `existsByCrise` + constraint `UNIQUE` em `crise_id` no banco.

7.3 UsuarioService — CRUD administrativo com salvaguardas

- **E-mail único** na criação e na edição (excluindo o próprio registro).
- **Senha**: quando informada, tem no mínimo 8 caracteres (Bean Validation) e é hashada em BCrypt, zerando `mustChangePassword` ; na edição, vazia/nula mantém a senha atual.
- **Primeiro acesso**: no `create` **sem senha**, o BO gera uma senha temporária (`PasswordGenerator`), marca `mustChangePassword=true` e envia por e-mail (`EmailService.enviarSenhaTemporaria`). O envio é resiliente: se o SMTP falhar, a conta fica criada e o admin pode reenviar via `reset-senha` .
- **Perfil estendido**: `create` e `update` também gravam `cargo` e `instituicaoId` (além de `departamentoId`).
- **Auto-proteção**: não é possível desativar o próprio usuário.

```
@Transactional
public void deactivate(Long id) {
    User user = getOrThrow(id);
    Long atual = Long.valueOf(jsonWebToken.getSubject());
    if (user.id.equals(atual))
        throw new BadRequestException("Você não pode desativar o próprio usuário.");
    user.active = false; // soft-delete
}
```

7.4 DashboardService — agregação

```
public DashboardResponse resumo() {
    return new DashboardResponse(
        criseRepository.count(),
        criseRepository.countByStatus(StatusCrise.ABERTA),
        criseRepository.countByStatus(StatusCrise.EM_ANDAMENTO),
        criseRepository.countByStatus(StatusCrise.RESOLVIDA),
        criseRepository.countByNivel(NivelCriticidade.CRITICO));
}
```

7.5 Serviços de referência (Campus, Departamento, Instituicao, TipoCrise, CenarioCrise)

Compartilham a mesma forma: `listAll` , `findById` , `create` (define `active=true`), `update` (muta a entidade managed) e `deactivate` (soft-delete). Exemplo do `CampusService.update` — sem `persist` :

```

@Transactional
public CampusResponse update(Long id, CampusRequest request) {
    Campus campus = getOrThrow(id);           // entidade gerenciada
    campus.nome = request.nome();
    campus.instituicaoId = request.instituicaoId();
    return CampusResponse.from(campus); // flush automático no fim da transação
}

```

7.6 AuthService — perfil e recuperação de senha

Além de emitir o token no login (ver §10), o `AuthService` concentra a "Área do usuário": consulta de perfil, troca de senha autenticada e o fluxo público de recuperação por e-mail.

`getProfile()` — devolve um `ProfileResponse` com os nomes de departamento e instituição já resolvidos, além do `cargo` :

```

public ProfileResponse getProfile() {
    User user = usuarioAutenticado();
    String departamentoNome = user.departamentoId == null ? null
        : departamentoRepository.findByIdOptional(user.departamentoId).map(d → d.nome).orElse(null);
    String instituicaoNome = user.instituicaoId == null ? null
        : instituicaoRepository.findByIdOptional(user.instituicaoId).map(i → i.nome).orElse(null);
    return new ProfileResponse(user.id, user.name, user.email, user.perfil, user.cargo,
        user.departamentoId, departamentoNome, user.instituicaoId, instituicaoNome);
}

```

`changePassword()` — valida a senha atual, exige que a nova seja diferente, zera a flag de troca obrigatória e reemite o token (o controller regrava o cookie `jwt`):

```

@Transactional
public String changePassword(String currentPassword, String newPassword) {
    // ... carrega o usuário do subject do JWT ...
    if (!BcryptUtil.matches(currentPassword, user.password))
        throw new BadRequestException("Senha atual incorreta.");
    if (BcryptUtil.matches(newPassword, user.password))
        throw new BadRequestException("A nova senha deve ser diferente da atual.");
    user.password = BcryptUtil.bcryptHash(newPassword);
    user.mustChangePassword = false;           // primeira troca zera a flag
    return issueToken(user);                  // reemite o cookie jwt
}

```

`requestPasswordReset(email)` — gera um token aleatório **Base64url de 32 bytes**, guarda apenas o **SHA-256** do token (nunca o valor cru) com expiração de 30 minutos e dispara o e-mail com o link. A resposta ao cliente é sempre neutra — **não revela se o e-mail existe** (anti-enumeração de contas):

```

@Transactional
public void requestPasswordReset(String email) {
    userRepository.findByEmail(email).filter(u → u.active).ifPresent(user → {
        String token = gerarToken();           // 32 bytes aleatórios → Base64url
        user.resetToken = sha256(token);       // guarda só o HASH do token
        user.resetTokenExpiresAt = LocalDateTime.now().plusMinutes(30);
        emailService.enviarLinkRecuperacao(user.name, user.email, token, 30);
    });
    // sem else / sem exceção: o chamador recebe sempre a mesma resposta
}

```

`resetPassword(token, novaSenha)` — localiza o usuário pelo hash do token, valida a expiração, troca a senha (zerando também `mustChangePassword`) e limpa o token (uso único):

```
@Transactional
public void resetPassword(String token, String newPassword) {
    User user = userRepository.findByResetToken(sha256(token))
        .filter(u → u.resetTokenExpiresAt ≠ null
            && u.resetTokenExpiresAt.isAfter(LocalDateTime.now()))
        .orElseThrow(() → new BadRequestException(
            "Link de recuperação inválido ou expirado. Solicite um novo."));
    user.password = BcryptUtil.bcryptHash(newPassword);
    user.mustChangePassword = false;
    user.resetToken = null; user.resetTokenExpiresAt = null; // uso único
}
```

Por que guardar só o hash? Se o banco vaziar, o valor armazenado (SHA-256) não permite reconstruir o link válido — o token em claro só existiu no e-mail do usuário. Combinado à expiração curta e ao uso único, isso limita a janela de ataque.

8. Camada de controllers (JAX-RS)

Fino por design: recebem HTTP, validam com `@Valid`, aplicam segurança e delegam ao service. A anotação de classe define a política padrão; métodos podem endurecê-la.

PADRÃO — CRISECONTROLLER

```
@Path("/api/crises") @Authenticated
@Consumes(APPLICATION_JSON) @Produces(APPLICATION_JSON)
public class CriseController {
    @Inject CriseService criseService;

    @GET public List<CriseResponse> listAll() { return criseService.listAll(); }

    @POST @RolesAllowed({"ADMIN", "COMUM"}) // COMUM também cria crises
    public Response create(@Valid CriseRequest request) {
        return Response.status(CREATED).entity(criseService.create(request)).build();
    }

    @PATCH @Path("/{id}/status") @RolesAllowed("ADMIN")
    public CriseResponse changeStatus(@PathParam("id") Long id, @Valid StatusUpdateRequest r) { ... }

    @POST @Path("/{id}/acoes") @RolesAllowed("ADMIN") // registrar ação virou ADMIN-only
    public Response addAcao(...) { ... }
}
```

POLÍTICA DE ACESSO — RESUMO

Padrão do sistema: **leituras liberadas a qualquer autenticado; escritas só ADMIN**. Exceção: **criar crise** (ADMIN+COMUM). Crises e ações são **filtradas por posse** para o COMUM — ele só enxerga/acessa as próprias. **Registrar ação** e **relatórios** passaram a ser exclusivos do ADMIN. Usuários e auditoria seguem inteiramente ADMIN.

MAPA COMPLETO DE ENDPOINTS

Método · Rota	Acesso	Função
POST /auth/login	PÚBLICO	Autentica, seta cookie <code>jwt</code>
POST /auth/logout	AUTH	Expira o cookie
GET /auth/me	AUTH	Usuário da sessão
GET /auth/perfil	AUTH	Dados do perfil com nomes resolvidos
POST /auth/change-password	AUTH	Troca a própria senha (reemite o cookie)
POST /auth/forgot-password	PÚBLICO	Inicia recuperação (resposta neutra 204)
POST /auth/reset-password	PÚBLICO	Redefine a senha via token
GET /api/dashboard/resumo	AUTH	Indicadores
GET /api/crises · /{id}	AUTH	Lista / detalha (ADMIN vê todas; COMUM só as próprias)
POST /api/crises	ADMIN+COMUM	Cria (nasce ABERTA; grava o dono)
PATCH /api/crises/{id}/status	ADMIN	Avança status
GET /api/crises/{id}/acoes	AUTH	Lista ações (timeline)
POST /api/crises/{id}/acoes	ADMIN	Registra ação
GET /api/relatorios · /{id}	ADMIN	Lista / detalha
POST /api/relatorios	ADMIN	Cria relatório
GET/POST/PUT/DELETE /api/usuarios	ADMIN	CRUD de usuários (+ <code>reset-senha</code>)
GET /api/auditoria · /facets	ADMIN	Logs + filtros
GET /api/{ref} · /{id}	AUTH	Lista / detalha referência
POST/PUT/DELETE /api/{ref}	ADMIN	CRUD referência (soft-delete)

`{ref}` ∈ departamentos · tipos-crise · instituicoes · campi · cenarios

9. DTOs e validação (Bean Validation)

Entrada e saída são `record`s. Os `*Request` declaram as regras de validação, disparadas pelo `@Valid` no controller; violações viram `400` automaticamente.

```
public record CriseRequest(
    @NotBlank(message="O título é obrigatório") @Size(max=200) String titulo,
    @NotBlank @Size(max=1000) String descricao,
    @NotNull NivelCriticidade nivel,
    @NotNull Long tipoCriseId,
    @NotNull Long campusId,
    Long cenarioId,           // opcional (sem @NotNull)
    @NotNull Long departamentoId) {}
```

Os `*Response` convertem a entidade via um método fábrica estático, garantindo que a entidade JPA nunca é serializada diretamente (e a senha jamais sai). O `CriseResponse` ganhou `createdBy` e `createdByName`; o `AcaoCriseResponse` ganhou `usuarioNome` — nomes resolvidos no BO:

```
public record CriseResponse(Long id, String titulo, ..., Long createdBy,
    String createdByName, boolean active, LocalDateTime createdAt) {
    public static CriseResponse from(Crise c, String createdByName) {
        return new CriseResponse(c.id, c.titulo, ..., c.createdBy, createdByName, c.active, c.createdAt);
    }
}
```

UsuarioRequest ilustra validações compostas: `@Email`, `@Size(min=8,max=100)` na senha (opcional na edição), `@NotNull` no perfil e `@Size(max=100)` no cargo. O `UsuarioResponse` expõe id, name, email, perfil, departamentoid, instituicaoid, cargo, active, mustChangePassword, createdAt — **nunca** a senha.

Política de senha (backend): mínimo de **8 caracteres**, validado por Bean Validation em `ChangePasswordRequest`, `ResetPasswordRequest` e `UsuarioRequest` (todos `@Size(min=8, max=100)`).

10. Segurança — JWT em cookie, passo a passo

O token não trafega em `Authorization: Bearer`; ele é lido de um **cookie HttpOnly** chamado `jwt`. Como o JavaScript não acessa cookies HttpOnly, o token fica protegido contra roubo por XSS.

Emissão do token — `AuthService.login`

```
public LoginResult login(String email, String password) {
    User user = userRepository.findByEmail(email).orElseThrow(InvalidCredentialsException::new);
    if (!user.active || !BcryptUtil.matches(password, user.password))
        throw new InvalidCredentialsException();
    return new LoginResult(issueToken(user), user.mustChangePassword()); // front decide se força troca
}

// issueToken(user):
Jwt.issuer(issuer)
    .subject(user.id.toString()) // id → "quem sou"
    .claim("name", user.name)
    .claim("email", user.email)
    .groups(user.perfil.name()) // "ADMIN"/"COMUM" → role
    .expiresIn(TOKEN_EXPIRATION_SECONDS) // 3600s = 1 hora
    .sign(); // assina com privateKey.pem (RS256)
```

Entrega no cookie — `AuthController` + `AuthCookieFactory`

```
// AuthController.login
return Response.ok(new LoginResponse(true, "Login realizado com sucesso", result.mustChangePassword()))
    .cookie(authCookieFactory.buildLoginCookie(token, TOKEN_EXPIRATION_SECONDS)).build();

// AuthCookieFactory - atributos de segurança do cookie
new NewCookie.Builder("jwt")
    .value(token).path("/")
    .httpOnly(true) // inacessível ao JS
    .sameSite(NewCookie.SameSite.LAX) // mitiga CSRF
    .maxAge((int) maxAgeSeconds).build();
```

Fluxo completo

1 Cliente faz `POST /auth/login` (rota `@PermitAll`). 2 Servidor valida BCrypt e assina o JWT. 3 Devolve o token no cookie `jwt` HttpOnly. 4 Browser reenvia o cookie automaticamente a cada requisição. 5 SmallRye extrai/verifica o token do cookie; `@Authenticated` / `@RolesAllowed` autorizam. 6 `POST /auth/logout` devolve o cookie com `maxAge=0`, expirando-o.

Camadas de autorização

Mecanismo	Onde	Efeito
@PermitAll	login, forgot-password, reset-password	Acesso anônimo
@Authenticated	reads, /me, /perfil, change-password, dashboard	Qualquer perfil logado
@RolesAllowed("ADMIN")	escritas, registrar ação, relatórios, usuários, auditoria	Só administrador (claim groups)
@RolesAllowed({"ADMIN", "COMUM"})	criar crise	Ambos os perfis
Regra no BO	CriseService.getAcessivel	COMUM só acessa as próprias crises (403)
Regra no BO	CriseService.addAcao	COMUM restrito ao próprio departamento (403, defesa em profundidade)
Regra no BO	UsuarioService.deactivate	Bloqueia auto-desativação (400)

Política de senha: o backend exige no mínimo **8 caracteres** em toda definição/troca de senha (`ChangePasswordRequest` , `ResetPasswordRequest` , `UsuarioRequest`). O token de recuperação é guardado só como **SHA-256**, com expiração de 30 min e uso único.

11. Auditoria automática por filtro de resposta

Toda ação de modificação é registrada de forma transparente por um `ContainerResponseFilter` — os services não precisam chamar auditoria manualmente. Ele roda na *resposta* para capturar o status HTTP.

```
@Provider
public class AuditoriaLogFilter implements ContainerResponseFilter {
    @Override
    public void filter(ContainerRequestContext req, ContainerResponseContext resp) {
        if (!isModificacao(req.getMethod())) return; // só POST/PUT/PATCH/DELETE
        String subject = jwt.getSubject();
        if (subject == null || subject.isBlank()) return; // login anônimo não é auditado
        String entidade = extrairEntidade(normalize(req.getUriInfo().getPath()));
        if ("auth".equals(entidade) || "auditoria".equals(entidade)) return; // evita ruído
        try {
            auditoriaService.registrar(Long.valueOf(subject), nomeDoToken(subject),
                acaoDe(req.getMethod()), req.getMethod(), entidade,
                "/" + path, resp.getStatus());
        } catch (RuntimeException e) {
            LOG.warnf(e, "Falha ao registrar auditoria ... "); // nunca quebra a requisição
        }
    }
    // POST→CRIAR, PUT/PATCH→ATUALIZAR, DELETE→EXCLUIR
}
```

Transação própria: o `AuditoriaService.registrar` é `@Transactional(REQUIRES_NEW)`, porque o filtro executa *depois* que a transação da requisição original já foi encerrada. Assim o log é gravado numa transação independente e uma falha de auditoria não desfaz (nem depende de) a operação principal.

O que fica registrado em cada linha de `auditoria_logs`: quem (id + nome do token), ação amigável, método HTTP, entidade derivada do path, recurso completo, status HTTP e data/hora. Índices em `data_hora` DESC e `entidade_afetada` aceleram as consultas com filtro. Rotas de `auth` (incluindo troca/recuperação de senha) são deliberadamente ignoradas para evitar ruído.

12. Tratamento de exceções

Situação	Exceção	HTTP
Login inválido (e-mail, inativo, senha)	<code>InvalidCredentialsException</code> + <code>Mapper</code>	401 com JSON <code>LoginResponse(false, msg)</code>
Entidade inexistente	<code>NotFoundException</code>	404
Regra violada (crise encerrada, e-mail duplicado, relatório duplicado, auto-desativação, status inválido, senha atual incorreta, nova senha igual à atual, link de recuperação inválido/expirado)	<code>BadRequestException</code>	400
COMUM acessando crise que não é sua / fora do departamento	<code>ForbiddenException</code>	403
Falha de validação de DTO	Bean Validation	400 automático

```
@Provider
public class InvalidCredentialsExceptionMapper
    implements ExceptionMapper<InvalidCredentialsException> {
    @Override public Response toResponse(InvalidCredentialsException ex) {
        return Response.status(UNAUTHORIZED).type(APPLICATION_JSON)
            .entity(new LoginResponse(false, ex.getMessage())).build();
    }
}
```

As demais exceções (`NotFound` , `BadRequest` , `Forbidden`) usam os mappers embutidos do Quarkus/RESTEasy — só o caso de credenciais tem mapper próprio, para devolver um corpo JSON amigável.

13. Migrações Flyway (V1–V10)

O schema evolui de forma versionada e reproduzível. As migrações rodam em ordem no start.

Versão	Conteúdo
V1	users (email UNIQUE). Ainda sem perfil/departamento.
V2	departamentos e tipos_crise + seeds ("Coordenação de TI"/CTI e "Tecnologia").
V3	instituicoes , campi (FK→instituicoes), cenarios_crise (FK→tipos_crise, departamentos).
V4	crises com FKs (cenario_id nullable).
V5	acoes_crise (FK→crises, users).
V6	relatorios_crise — crise_id UNIQUE (1 relatório por crise).
V7	Adiciona perfil (default 'COMUM') e departamento_id em users ; promove o admin a ADMIN.
V8	auditoria_logs + índices data_hora DESC e entidade_afetada .
V9	Adiciona must_change_password BOOLEAN NOT NULL DEFAULT FALSE em users (força a troca da senha temporária no 1º acesso).
V10	Posse e perfil estendido : crises.created_by (FK→users) + índice idx_crises_created_by ; em users , cargo , instituicao_id (FK→instituicoes), reset_token e reset_token_expires_at (recuperação de senha).

```
-- V10: dono da crise + dados de perfil + token de recuperação
ALTER TABLE crises ADD COLUMN created_by BIGINT REFERENCES users (id);
CREATE INDEX idx_crises_created_by ON crises (created_by);

ALTER TABLE users ADD COLUMN cargo          VARCHAR(100);
ALTER TABLE users ADD COLUMN instituicao_id   BIGINT REFERENCES instituicoes (id);
ALTER TABLE users ADD COLUMN reset_token     VARCHAR(64);      -- SHA-256 (hex)
ALTER TABLE users ADD COLUMN reset_token_expires_at TIMESTAMP;
```

14. Ciclo de vida completo de uma crise (walkthrough)

Juntando todas as camadas, veja o caminho de ponta a ponta:

1 LOGIN

POST /auth/login → AuthService valida BCrypt → JWT no cookie jwt

2 ABERTURA (ADMIN ou COMUM)

POST /api/crises → @RolesAllowed({ADMIN,COMUM}) → CriseService.create()
status = ABERTA (forçado) ; createdBy = autor (posse) ; AuditoriaLogFilter grava CRIAR/crises/201

3 ATUAÇÃO (ADMIN)

POST /api/crises/{id}/acoes → @RolesAllowed(ADMIN) : só ADMIN registra ação
nova AcaoCrise com autor = subject do JWT ; auditado
(COMUM lê a timeline em GET /acoes das próprias crises)

4 EVOLUÇÃO DO STATUS (ADMIN)

PATCH /api/crises/{id}/status
ABERTA → EM_ANDAMENTO → RESOLVIDA → ENCERRADA
(ENCERRADA é terminal: bloqueia novas mudanças → 400)

5 RELATÓRIO FINAL (ADMIN)

POST /api/relatorios → exige RESOLVIDA/ENCERRADA e 1 por crise

6 RASTREABILIDADE

Cada POST/PUT/PATCH/DELETE gerou 1 linha em auditoria_logs
consultável em /api/auditoria (só ADMIN)

15. Área do usuário (perfil COMUM)

A "Área do usuário" reorganiza a experiência do perfil `COMUM` em torno de três pilares: onboarding por senha temporária, recuperação de senha autônoma e **posse das crises** que ele mesmo cadastra — com um endurecimento deliberado das permissões.

15.1 Primeiro acesso — senha temporária

Quando o admin cria um usuário **sem informar senha**, o `UsuarioService` gera uma senha temporária (`PasswordGenerator`), grava `mustChangePassword=true` e a envia por e-mail (`EmailService.enviarSenhaTemporaria`). No login, o backend devolve `mustChangePassword` no `LoginResponse` ; o front trata isso como bloqueio até a primeira troca. A primeira chamada a `POST /auth/change-password` zera a flag e reemite o cookie `jwt` .

15.2 Recuperação de senha ("Esqueci minha senha")

Fluxo público em duas etapas, ambas `@PermitAll` :

- `POST /auth/forgot-password` — recebe o e-mail e responde sempre **204** (resposta neutra, anti-enumeração). Se o e-mail existir e a conta estiver ativa, gera um token aleatório, guarda apenas o **SHA-256** dele com expiração de **30 min** e dispara o e-mail com o link `/redefinir-senha?token=...` (`EmailService.enviarLinkRecuperacao`).
- `POST /auth/reset-password` — recebe `token` + `newPassword` , valida o hash e a expiração, troca a senha e **limpa o token** (uso único).

15.3 Posse de crises

A coluna `crises.created_by` (V10) marca o dono de cada crise. O `ADMIN` continua vendo e acessando todas; o `COMUM` só enxerga (`listByCreator`) e só acessa (`getAcessivel`) as que ele mesmo cadastrou — tentar abrir a crise de outra pessoa resulta em **403** . Crises anteriores à V10 têm `created_by` nulo (daí o cuidado com `HashMap` na resolução de nomes, §7.1).

15.4 Endurecimento das permissões do COMUM

O `COMUM` deixou de **registrar ações** (`POST /api/crises/{id}/acoes` virou `@RolesAllowed("ADMIN")`) e não acessa **relatórios** (o `RelatorioCriseController` inteiro é `@RolesAllowed("ADMIN")` , leitura e criação). Ele continua podendo **criar crises** e **ler a linha do tempo** (`GET /api/crises/{id}/acoes` segue `@Authenticated`) das próprias crises.

15.5 Política de senha

Toda definição/troca de senha exige no mínimo **8 caracteres** no backend (`ChangePasswordRequest` , `ResetPasswordRequest` , `UsuarioRequest`).

15.6 Mini-walkthrough

PRIMEIRO ACESSO

- 1 ADMIN cria usuário SEM senha → `UsuarioService` gera senha temporária
`mustChangePassword = true` · e-mail: `enviarSenhaTemporaria(...)`
- 2 Usuário faz login → `LoginResponse.mustChangePassword = true` (front bloqueia)
- 3 `POST /auth/change-password` (senha atual = temporária, nova ≠ atual)
zera `mustChangePassword` · reemite o cookie `jwt`

RECUPERAÇÃO DE SENHA

- 1 `POST /auth/forgot-password { email }` → **204** (resposta neutra)
token aleatório → e-mail `/redefinir-senha?token=...`
banco guarda `SHA-256(token)` + expiração (30 min)
- 2 `POST /auth/reset-password { token, newPassword }`
valida hash + expiração → troca a senha, limpa o token (uso único)

